

Website QA, explained simply

How to add an automatic website checker to an Astro site — what it does, how to set it up, and how to read what it tells you. Written for people who are not developers.

WHAT IT IS

A robot that checks your website before it goes live

TIME, FIRST SITE

About 1–2 hours

WHAT IT COSTS

Nothing — it runs on GitHub for free

TIME, EVERY SITE AFTER

About 10 minutes

What's in this guide

- 1 What this thing actually does
- 2 Before you start
- 3 Adding it to a website that already exists
- 4 Telling it about your website
- 5 Running it on your own computer
- 6 How to read the results
- 7 When the site is older and there's a lot to fix
- 8 Turning on the lock
- 9 Proving it actually works
- 10 Things that go wrong, and what to do
- 11 Words you'll see, explained

1 What this thing actually does

Think of it as a very thorough, very fast assistant who checks every page of a website before a client ever sees it — and who is allowed to say "no, this isn't going live yet."

The problem it solves

Most things that embarrass a web team on launch day are not clever or subtle. They are things like:

- A link that goes to a page that no longer exists
- A photo that doesn't load
- The *previous* client's business name still sitting in the footer, because this site was copied from theirs
- The contact form quietly disappearing during a redesign
- A page that scrolls sideways on a phone

Every one of these is findable by a machine. They get missed because checking twenty pages across five screen sizes by hand is dull work, and dull work gets skipped when a deadline is close.

IN ONE SENTENCE

It builds your website, opens it in real web browsers, clicks around, and refuses to let a broken version go live.

What it checks

Thirty-four checks, grouped into seven areas:

AREA	IN PLAIN ENGLISH
Pages & search	Do the important pages load? Does each one have a title and description so Google can list it properly? Is the "page not found" page intact?
Identity	Is the right business name on the site? Is the <i>wrong</i> one anywhere? Are the phone number and email correct, and clickable?
Forms	Does the contact form still exist, and does it have the fields it needs?
Links	Does every link actually go somewhere?
Images	Do the pictures load? Are any stretched out of shape or enormous?
Layout	Does the page look right on a phone, a tablet and a desktop? Anything overlapping, cut off, or sliding off the side?
Clicking & accessibility	Do menus open and close? Can someone using only a keyboard get around? Is the text readable against its background?

What it does *not* do

It doesn't judge whether the design is nice, whether the writing is any good, or whether the site does what the client wanted. It checks that things *work*, not that they are *right*. A human still has to look at it.

How it fits into the day

- Someone changes the website and saves their work to GitHub
 - The robot wakes up automatically and starts checking
 - Finds something broken → the website does **not** go live
 - Everything fine → the website goes live by itself

Nobody has to remember to run it. That's the whole point.

2 Before you start

What you need

THING	WHY
An Astro website in GitHub	This is the website's filing cabinet, stored online
Admin access to it	You'll need to change settings, not just files
Node.js version 22 or newer	The software that runs the checks. Free
About 2 hours, once	Only for the first site. Later ones take minutes

CHECK YOUR NODE VERSION FIRST

Open a terminal and type `node --version`. If it says anything lower than `v22`, install a newer one from nodejs.org before going further. Astro refuses to build on older versions, and you'll waste time wondering why.

One thing to find out before you begin

Open your website's folder and look for where `package.json` lives. There are two common shapes:

SHAPE	WHAT IT MEANS FOR YOU
<code>package.json</code> at the top	The simple case. Follow the steps exactly as written
<code>package.json</code> inside a <code>site/</code> folder	Very common at Goodly. You'll need one extra step — see the box in section 3

3 Adding it to a website that already exists

This is the normal case. The website is already built, already live, and has been running for months or years. You are adding a safety net to something that already works — not building a new site.

NOTHING YOU DO HERE CHANGES THE WEBSITE ITSELF

The checker only *reads* the site. It adds its own files alongside yours, and it never edits your pages, your styling or your content. If you change your mind, section 3 ends with how to take it out again.

1 Get the kit onto your computer

You only ever do this once. It puts a copy of the checking tools on your machine.

```
git clone https://github.com/djavier0134/astro-qa-kit.git
```

Remember where it lands — you'll point at this folder from every site.

2 Get a copy of the website

If the site isn't already on your computer, download it from GitHub. Replace the address with your own site's:

```
git clone https://github.com/your-org/your-site.git
cd your-site
```

If you already have it, make sure you're starting from the current version rather than something from three weeks ago:

```
git checkout main
git pull
```

3 Make a safe space to work in

Never do setup work directly on the live version. Make a "branch" — a private copy you can experiment in without anyone noticing. Nothing you do on a branch can affect the real website until somebody deliberately merges it.

```
git checkout -b setup/qa-cicd
```

4 Copy the kit into your website

From inside your website's folder:

```
node /path/to/astro-qa-kit/install-qa.mjs .
```

If your `package.json` is inside a `site/` folder, use `site` instead of the dot:

```
node /path/to/astro-qa-kit/install-qa.mjs site
```

IMPORTANT IF YOU USED `SITE`

The installer will put a file at `site/.github/workflows/qa-deploy.yml`. **That is the wrong place.** GitHub only looks for these files at the very top of the project, so if you leave it there, the checker will never run at all — and everything will look fine, because there will be no errors. Move it up:

```
mkdir -p .github/workflows
mv site/.github/workflows/qa-deploy.yml .github/workflows/
```

Then open that file and ask a developer to add `working-directory: site` to it. This is the single most common way to end up with a checker that silently does nothing.

5 Check what it added

Run this to see exactly what changed:

```
git status --short
```

You should see a list like this, and nothing else:

WHAT APPEARED	WHAT IT IS
<code>scripts/qa-*.mjs</code>	The checking engine
<code>scripts/qa-config.json</code>	The settings file you'll edit next
<code>tests/</code>	Browser tests
<code>.github/workflows/</code>	Instructions for GitHub's robot
<code>package.json</code> (modified)	New shortcut commands added
<code>.gitignore</code> (modified)	Stops reports cluttering the project

None of your own files are touched. The installer refuses to overwrite anything that already exists, so your pages, images and styling are exactly as they were.

IF THE SITE ALREADY HAD ITS OWN CHECKS

Some older sites have a home-made `qa` command already. Because the installer won't overwrite it, you can end up with two different things both called "qa", which is confusing later. If `package.json` already had a `qa` entry, ask a developer to rename the old one to `qa:legacy` so there's no ambiguity about which one runs.

6 Install the extra tools

These are the robot's eyes — real web browsers it can drive.

```
npm install -D playwright@1.57.0 @playwright/test@1.57.0 @lhci/cli wait-on wrangler @axe-core/playwright
```

```
npx playwright install chromium firefox webkit
```

That second command downloads about 400 MB. It only happens once per computer.

IF YOU'RE ON WINDOWS AND THE DOWNLOAD FAILS

Some networks can't reach the servers these browsers come from. You don't need to fix it — just tell the checker to use the Chrome you already have installed. See section 5.

If you want to undo all of it

While you're still on your branch and haven't merged anything, you can throw the whole thing away and the website is untouched:

```
git checkout main  
git branch -D setup/qa-cicd
```

That deletes your branch and everything on it. The live site never saw any of it. This is why step 3 insists on working in a branch.

4 Telling it about your website

This is the only part that's different for each site, and it's the part worth taking your time over. Everything else is identical everywhere.

Open the file `scripts/qa-config.json`. It's a list of settings. Change the values between the quote marks; leave the rest alone.

The business details

```
"businessName": "Ammons Awnings & Handrails",
"businessNameAliases": ["Ammons Awnings", "Ammons"],
"forbiddenBusinessNames": ["Some Other Client", "Lorem Ipsum"],

"phone": "912-261-1069",
"email": "hello@example.com",
"emailDomain": "example.com",
```

THE MOST VALUABLE SETTING IN THE WHOLE FILE

`forbiddenBusinessNames`. Because sites get copied from one another, the previous client's name sometimes survives into the new site. Put the names of your *other* clients here. If one of them ever shows up on this website, the checker stops everything immediately. This single setting prevents the most embarrassing mistake in the business.

Which pages matter

```
"requiredPages": ["/", "/about", "/services", "/contact", "/privacy-policy"],
"formPages": ["/contact"],
```

These are the pages that *must* work. If any of them fails to load, nothing goes live.

SLASHES MATTER MORE THAN YOU'D THINK

Some sites use `/contact` and some use `/contact/` — with a slash at the end. These are treated as different addresses, and using the wrong one gives you a page-not-found error for a page that is perfectly fine. Open the real website, look at the address bar, and copy whichever form it uses.

Where things are on the page

```
"logoSelector": "header a.brand",
"headerSelector": "header",
"footerSelector": "footer",
```

These tell the checker where to find the logo, the top bar and the bottom bar. A developer can read these off the site in a couple of minutes. If they're wrong, you'll get confusing "logo not found" messages on every single page.

Leave these alone at first

`design.colorRules`, `design.noOverlapSelectors` and `visualRegression` all start switched off deliberately. Turn them on later, once the design is signed off. Turning them on early means spending your first week chasing complaints about things that aren't actually wrong.

5 Running it on your own computer

DO THIS BEFORE YOU UPLOAD ANYTHING

Get it working on your own machine first. Checking on your computer takes seconds. Checking online takes about eight minutes each time. Finding your first forty problems eight minutes at a time is a miserable afternoon.

You need two terminal windows.

Window one — start a local copy of the website and leave it running:

```
npm run build
npm run preview -- --host 127.0.0.1 --port 4321
```

Window two — run the checks:

```
npm run qa:all
```

While you're still fixing things, skip the two slowest checks:

```
npm run qa:all -- --skip lighthouse,crossbrowser
```

ON WINDOWS, IF IT SAYS "EXECUTABLE DOESN'T EXIST"

It couldn't download its own browser. Tell it to borrow Chrome instead. In PowerShell:

```
$env:QA_CHROMIUM_CHANNEL="chrome"; npm run qa:all -- --skip lighthouse,crossbrowser
```

Only do this on your own computer. The online version downloads its browsers without trouble.

Where the results appear

FILE	WHAT'S IN IT
qa-report.md	The readable report. Start here
qa-results/screenshots/	Pictures of any page that looked wrong

6 How to read the results

Every finding is one of three things:

LABEL	WHAT HAPPENS	WHAT IT MEANS
ERROR	Nothing goes live	Something is genuinely broken. Fix it
WARNING	Site still goes live	Worth looking at, not urgent
PASS	—	That check was fine

Only errors stop a release. You can have fifty warnings and still go live. That's deliberate — if warnings blocked things, you'd never ship anything.

What to do with each finding

IF THE FINDING IS...	THEN
A genuine problem with the site	Fix the website
A real rule you're not ready to enforce	Change it to <code>"warning"</code> in the settings
The checker misunderstanding something	Fix the settings file, not the website

RULES WORTH KEEPING STRICT FROM DAY ONE

Wrong business name, broken internal links, missing contact form, a required page that won't load, a page that scrolls sideways on a phone, a page accidentally hidden from Google, and a missing privacy policy. These are the ones that cause real damage.

Aim for zero errors, not zero warnings

Zero errors is the goal. Carrying warnings is completely normal and nothing to feel bad about. Resist the urge to switch rules off just to get a clean screen — a checker that's been quietened into silence protects nobody.

7 When the site is older and there's a lot to fix

The first time you point this at a website that has been live for years, it will find a lot. That is not a disaster and it is not a sign the site is bad. It is simply the first time anyone has counted.

Expect this, and don't panic

A mature site commonly turns up thirty or forty findings on the first run. Most will be warnings. A handful will be real. On the live client site we tested this with, the very first run found a genuine accessibility problem that had been shipped and never noticed — which is the point of the exercise.

THE MISTAKE TO AVOID

Do not try to fix everything before switching it on, and do not switch off all the rules to get a clean screen. Both end the same way: the checker gets abandoned. Aim for **zero errors** and accept the warnings for now.

A sensible order to work in

ORDER	DO THIS
First	Fix anything that is genuinely broken — dead links, missing forms, a wrong business name. These are usually few and quick
Second	For real problems you can't fix today, change them from <code>"error"</code> to <code>"warning"</code> . They stay visible but stop blocking releases
Third	Correct anything where the checker simply misunderstood the site — wrong addresses, wrong selectors. Fix the settings, not the website
Later	Work the warnings down over the following weeks, promoting rules back to <code>"error"</code> as you clear them

Softening rules on day one

These four are the usual suspects on an established site, and it is perfectly reasonable to start with them turned down:

```
"severity": {  
  "accessibilityMinor": "off",  
  "lazyLoading": "warning",  
  "typography": "warning",  
  "externalLink": "off"  
}
```

`"off"` means don't mention it. `"warning"` means tell me but don't block. `"error"` means stop everything.

KEEP THESE STRICT FROM THE VERY FIRST DAY

Wrong business name, broken internal links, missing contact form, a required page that won't load, sideways scrolling on a phone, a page hidden from Google by accident, and a missing privacy policy. These are the ones that cost money or embarrassment, and they're almost never hard to fix.

8 Turning on the lock

Up to now the checker reports problems but can't actually stop anything. This is where you give it teeth.

1 Upload your work

```
git add .
git commit -m "Add automatic website checking"
git push -u origin setup/qa-cicd
```

Then on GitHub, open a "pull request" — a request to merge your work in.

2 Watch it run

Go to the **Actions** tab. You'll see it working through the checks. Scroll to the bottom of the page for the full report.

First time online usually turns up a few new things. Different computer, slightly different results. Normal.

3 Lock the main version

Settings → Rules → Rulesets → New branch ruleset. Name it *Protect main*, set it to **Active**, target the default branch, and tick:

- Require a pull request before merging
- Require status checks to pass → choose **Website QA**
- Block force pushes

CAN'T FIND "WEBSITE QA" IN THE LIST?

It only appears after the check has run at least once. Let step 2 finish, then come back.

THE STEP PEOPLE SKIP — AND IT UNDOES EVERYTHING

If your hosting (Cloudflare Pages) is connected directly to GitHub, it builds and publishes the site *by itself* whenever the main version changes — without ever looking at the checker's verdict. You would have a lock on the front door and an open window beside it. Either disconnect that direct connection and let the checker handle publishing, or accept that the checker is only giving advice. Ask a developer which applies to your site.

9 Proving it actually works

Don't trust a safety net you've never watched catch anything. Break things on purpose, on your branch, and check it notices. Put each one back afterwards.

BREAK THIS ON PURPOSE	YOU SHOULD SEE
Add a link to a page that doesn't exist	Error naming the bad link
Point a photo at a missing file	Error naming the image
Delete the contact form	Error: required form not found
Put another client's name on a page	Error naming the word and the page
Add something very wide to a page	Error on phone and tablet, plus a screenshot
Break the mobile menu button	Error: the menu didn't open

ON A REAL CLIENT'S WEBSITE

Only ever do these on your own branch, never on the live version — and only after the hosting question in section 8 is settled. Otherwise you risk publishing a deliberately broken site to a real customer.

10 Things that go wrong, and what to do

WHAT YOU SEE	WHAT'S HAPPENING
Checks never run at all, no errors anywhere	The workflow file is in the wrong folder. See the red box in section 3. This is the dangerous one, because it looks like success
<code>Node.js v20 is not supported by Astro</code>	Node is too old. It needs version 22 or newer
Pages you know exist are reported as missing	Almost always the trailing-slash problem in section 4
<code>Executable doesn't exist</code>	Browsers didn't download. On Windows, use the Chrome workaround in section 5
You fixed something, but the same error keeps appearing	You're probably looking at an old run, or pressed "Re-run jobs" on one. Re-running re-tests the <i>old</i> version. Check the run matches your newest change
Dozens of accessibility complaints	Normal on an older site. Set <code>accessibilityMinor</code> to <code>"off"</code> , deal with the serious ones first
Performance scores wobble between runs	Shared machines vary. Lower the threshold slightly, or raise the number of runs so it takes an average
The site went live even though checks failed	The hosting is still publishing on its own. Go back to the red box in section 8

11 Words you'll see, explained

WORD	WHAT IT MEANS
Repository (repo)	The website's filing cabinet, stored on GitHub
Branch	A private copy where you can work without affecting the live site
Main	The real version. What's on main is what customers get
Commit	A saved change, with a note about what you did
Push	Uploading your saved changes to GitHub
Pull request	Asking for your branch to be merged into main
Merge	Folding your branch into main. This is the moment it becomes real
Deploy	Publishing the website so the public can see it
GitHub Actions	GitHub's robot. It's what runs the checks
Workflow	The instructions telling that robot what to do
Build	Turning the website's source files into actual web pages
Artifact	Files saved from a run — reports, screenshots — for you to download
Lighthouse	Google's tool for scoring speed, accessibility and search-readiness
Accessibility (a11y)	Whether people with disabilities can use the site
Viewport	Screen size — phone, tablet, laptop, desktop
Selector	A short piece of text that points at part of a page, like the logo

IF YOU REMEMBER NOTHING ELSE

Get it working on your own computer before uploading. Only errors stop a release. And check that your hosting isn't publishing the site behind the checker's back — otherwise none of this protects you.